Name: Prasana Shangloo
Roll No.: 23102A0076
Branch: CMPN A

```
options(repos = c(CRAN =
"https://packagemanager.posit.co/cran/__linux__/jammy/latest"))
pkgs <- c("data.table", "ggplot2", "foreach", "doParallel", "microbenchmark",
          "purrr", "lubridate", "arrow", "scales")
new <- setdiff(pkgs, rownames(installed.packages()))
if (length(new)) install.packages(new)
invisible(lapply(pkgs, library, character.only = TRUE))

set.seed(42)
options(width = 120, datatable.print.nrows = 15)
cat("Logical cores available:", parallel::detectCores(), "\n")
cat("data.table threads     :", getDTthreads(), "\n")
R.version.string
```

```
Installing packages into ‘/usr/local/lib/R/site-library’
(as ‘lib’ is unspecified)

also installing the dependencies ‘iterators’, ‘assertthat’



Attaching package: ‘data.table’


The following object is masked from ‘package:base’:

    %notin%


Loading required package: iterators

Loading required package: parallel


Attaching package: ‘purrr’


The following objects are masked from ‘package:foreach’:

    accumulate, when


The following object is masked from ‘package:data.table’:

    transpose



Attaching package: ‘lubridate’


The following objects are masked from ‘package:data.table’:

    hour, isoweek, isoyear, mday, minute, month, quarter, second, wday,
    week, yday, year


The following objects are masked from ‘package:base’:

    date, intersect, setdiff, union



Attaching package: ‘arrow’


The following object is masked from ‘package:lubridate’:

    duration


The following object is masked from ‘package:utils’:

    timestamp
```

```
Attaching package: ‘scales’


The following object is masked from ‘package:purrr’:

    discard
```

```
Logical cores available: 2
data.table threads     : 1
```

'R version 4.6.1 (2026-06-24)'

```r
mem_time <- function(expr) {
  invisible(gc(reset = TRUE))
  t <- system.time(res <- force(expr))
  g <- gc()
  list(elapsed = unname(t["elapsed"]), peak_mb = sum(g[, 6]), result = res)
}

mb_tab <- function(mb, label) {
  s <- as.data.table(summary(mb, unit = "ms"))
  data.table(Benchmark = label, Approach = as.character(s$expr),
             Min_ms = round(s$min, 2), Median_ms = round(s$median, 2),
             Mean_ms = round(s$mean, 2), Max_ms = round(s$max, 2))
}
```

```r
dir.create("data", showWarnings = FALSE)
base_url <- "https://d37ci6vzurychx.cloudfront.net"
months   <- sprintf("2024-%02d", 1:3)

keep_cols <- c("tpep_pickup_datetime", "tpep_dropoff_datetime", "PULocationID",
"DOLocationID",
               "passenger_count", "trip_distance", "payment_type", "fare_amount",
               "tip_amount", "tolls_amount", "total_amount")

read_month <- function(m) {
  f <- file.path("data", sprintf("yellow_tripdata_%s.parquet", m))
  if (!file.exists(f))
    download.file(sprintf("%s/trip-data/yellow_tripdata_%s.parquet", base_url, m),
                  f, mode = "wb", quiet = TRUE)
  as.data.table(arrow::read_parquet(f, col_select = keep_cols))
}

t_import <- system.time(raw <- rbindlist(lapply(months, read_month)))
cat("Import time (s):", t_import["elapsed"], "\n")
```

```
Warning message:
“Using an external vector in selections was deprecated in tidyselect 1.1.0.
i Please use `all_of()` or `any_of()` instead.
  # Was:
  data %>% select(keep_cols)

  # Now:
  data %>% select(all_of(keep_cols))

See <https://tidyselect.r-lib.org/reference/faq-external-vector.html>.”
```

```
Import time (s): 8.883
```

```
cat("Dimensions (rows x cols):", dim(raw), "\n")
cat("Memory used by raw data :", format(object.size(raw), units = "MB"), "\n\n")
str(raw)
print(head(raw))
print(summary(raw))
```

```
Dimensions (rows x cols): 9554778 11
Memory used by raw data : 656.1 Mb

Classes ‘data.table’ and 'data.frame':  9554778 obs. of  11 variables:
 $ tpep_pickup_datetime : POSIXct, format: "2024-01-01 00:57:55" "2024-01-01 00:0
3:00" "2024-01-01 00:17:06" "2024-01-01 00:36:38" ...
 $ tpep_dropoff_datetime: POSIXct, format: "2024-01-01 01:17:43" "2024-01-01 00:0
9:36" "2024-01-01 00:35:01" "2024-01-01 00:44:56" ...
 $ PULocationID         : int  186 140 236 79 211 148 138 246 161 113 ...
 $ DOLocationID         : int  79 236 79 211 148 141 181 231 261 113 ...
 $ passenger_count      : int  1 1 1 1 1 1 2 0 1 1 ...
 $ trip_distance        : num  1.72 1.8 4.7 1.4 0.8 ...
 $ payment_type         : int  2 1 1 1 1 1 1 2 2 2 ...
 $ fare_amount          : num  17.7 10 23.3 10 7.9 29.6 45.7 25.4 31 3 ...
 $ tip_amount           : num  0 3.75 3 2 3.2 6.9 10 0 0 0 ...
 $ tolls_amount         : num  0 0 0 0 0 0 0 0 0 0 ...
 $ total_amount         : num  22.7 18.8 31.3 17 16.1 ...
 - attr(*, ".internal.selfref")=<pointer: 0x59fb4a0163f0>
   tpep_pickup_datetime tpep_dropoff_datetime PULocationID DOLocationID passenger_
count trip_distance payment_type
                 <POSc>               <POSc>        <int>        <int>
<int>         <num>        <int>
1:  2024-01-01 00:57:55   2024-01-01 01:17:43          186           79
1          1.72            2
2:  2024-01-01 00:03:00   2024-01-01 00:09:36          140          236
1          1.80            1
3:  2024-01-01 00:17:06   2024-01-01 00:35:01          236           79
1          4.70            1
4:  2024-01-01 00:36:38   2024-01-01 00:44:56           79          211
1          1.40            1
5:  2024-01-01 00:46:51   2024-01-01 00:52:57          211          148
1          0.80            1
6:  2024-01-01 00:54:08   2024-01-01 01:26:31          148          141
1          4.70            1
   fare_amount tip_amount tolls_amount total_amount
         <num>      <num>        <num>        <num>
1:        17.7       0.00            0        22.70
2:        10.0       3.75            0        18.75
3:        23.3       3.00            0        31.30
4:        10.0       2.00            0        17.00
5:         7.9       3.20            0        16.10
6:        29.6       6.90            0        41.50
 tpep_pickup_datetime          tpep_dropoff_datetime          PULocationID    DOLo
cationID   passenger_count
 Min.   :2002-12-31 22:17:10   Min.   :2002-12-31 22:42:24   Min.   :  1.0   Min.
:  1.0   Min.   :0.000
 1st Qu.:2024-01-26 09:55:41   1st Qu.:2024-01-26 10:12:00   1st Qu.:132.0   1st Q
u.:114.0   1st Qu.:1.000
 Median :2024-02-18 15:38:32   Median :2024-02-18 15:55:02   Median :162.0   Media
n :162.0   Median :1.000
 Mean   :2024-02-17 17:20:27   Mean   :2024-02-17 17:36:34   Mean   :165.2   Mean
:164.2   Mean   :1.334
 3rd Qu.:2024-03-11 11:59:11   3rd Qu.:2024-03-11 12:16:17   3rd Qu.:234.0   3rd Q
u.:234.0   3rd Qu.:1.000
 Max.   :2024-04-01 00:34:55   Max.   :2024-04-02 18:08:46   Max.   :265.0   Max.
:265.0   Max.   :9.000

NAs    :751962
 trip_distance        payment_type    fare_amount        tip_amount        tolls_a
mount       total_amount
```

```
 Min.   :0.000e+00   Min.   :0.000   Min.   :-999.00   Min.   :-300.000   Min.   :
-84.3000   Min.   :-1000.00
 1st Qu.:1.000e+00   1st Qu.:1.000   1st Qu.:   8.60   1st Qu.:   0.000   1st Qu.:
0.0000   1st Qu.:   15.47
 Median :1.700e+00   Median :1.000   Median :  12.80   Median :   2.650   Median :
0.0000   Median :   20.20
 Mean   :4.042e+00   Mean   :1.121   Mean   :  18.33   Mean   :   3.271   Mean   :
0.5264   Mean   :   26.87
 3rd Qu.:3.190e+00   3rd Qu.:1.000   3rd Qu.:  20.50   3rd Qu.:   4.120   3rd Qu.:
0.0000   3rd Qu.:   28.92
 Max.   :3.127e+05   Max.   :4.000   Max.   :9792.00   Max.   : 999.990   Max.   :
163.0000   Max.   : 9792.00
```

```r
cat("Missing values per column:\n")
print(raw[, lapply(.SD, function(x) sum(is.na(x)))])
cat("\nExact duplicate rows:", sum(duplicated(raw)), "\n\n")

dur <- as.numeric(difftime(raw$tpep_dropoff_datetime, raw$tpep_pickup_datetime,
units = "mins"))
checks <- data.table(
  Rule = c("fare_amount <= 0", "trip_distance <= 0", "trip_distance >= 100",
"total_amount <= 0",
           "passenger_count outside 1-6", "duration < 1 min or > 180 min",
           "pickup outside Jan-Mar 2024", "tip or tolls negative"),
  Records = c(sum(raw$fare_amount <= 0, na.rm = TRUE),
              sum(raw$trip_distance <= 0, na.rm = TRUE),
              sum(raw$trip_distance >= 100, na.rm = TRUE),
              sum(raw$total_amount <= 0, na.rm = TRUE),
              sum(!(raw$passenger_count %between% c(1, 6)), na.rm = TRUE),
              sum(dur < 1 | dur > 180, na.rm = TRUE),
              sum(raw$tpep_pickup_datetime < as.POSIXct("2024-01-01", tz = "UTC")
|
                  raw$tpep_pickup_datetime >= as.POSIXct("2024-04-01", tz =
"UTC"), na.rm = TRUE),
              sum(raw$tip_amount < 0 | raw$tolls_amount < 0, na.rm = TRUE)))
print(checks)
rm(dur)
```

```
Missing values per column:
   tpep_pickup_datetime tpep_dropoff_datetime PULocationID DOLocationID passenger_
count trip_distance payment_type
                  <int>                 <int>        <int>        <int>
<int>         <int>        <int>
1:                    0                     0            0            0          7
51962             0            0
   fare_amount tip_amount tolls_amount total_amount
         <int>      <int>        <int>        <int>
1:           0          0            0            0

Exact duplicate rows: 1

                           Rule Records
                         <char>   <int>
1:              fare_amount <= 0  139596
2:            trip_distance <= 0  215764
3:          trip_distance >= 100     252
4:             total_amount <= 0  117187
5:   passenger_count outside 1-6  106030
6: duration < 1 min or > 180 min  115627
7:   pickup outside Jan-Mar 2024      21
8:         tip or tolls negative    7896
```

```r
n_raw     <- nrow(raw)
raw_mb    <- as.numeric(object.size(raw)) / 1024^2

dt <- unique(raw)
rm(raw); invisible(gc())
n_dedup <- nrow(dt)

dt[, duration_min := as.numeric(difftime(tpep_dropoff_datetime,
tpep_pickup_datetime, units = "mins"))]

dt <- dt[!is.na(passenger_count) & passenger_count >= 1 & passenger_count <= 6 &
         trip_distance > 0 & trip_distance < 100 &
         fare_amount > 0 & fare_amount < 500 &
         total_amount > 0 & total_amount < 1000 &
         tip_amount >= 0 & tolls_amount >= 0 &
         duration_min >= 1 & duration_min <= 180 &
         tpep_pickup_datetime >= as.POSIXct("2024-01-01", tz = "UTC") &
         tpep_pickup_datetime <  as.POSIXct("2024-04-01", tz = "UTC") &
         PULocationID %between% c(1, 265) & DOLocationID %between% c(1, 265)]

dt[, `:=`(PULocationID    = as.integer(PULocationID),
          DOLocationID    = as.integer(DOLocationID),
          passenger_count = as.integer(passenger_count),
          payment_type    = factor(payment_type, levels = 0:6,
                                   labels = c("Flex fare", "Credit card", "Cash",
"No charge",
                                              "Dispute", "Unknown", "Voided")))]

dt[, `:=`(hour      = hour(tpep_pickup_datetime),
          day       = mday(tpep_pickup_datetime),
          dow       = factor(wday(tpep_pickup_datetime), levels = 1:7,
                             labels = c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri",
"Sat")),
          month     = month(tpep_pickup_datetime),
          month_lbl = factor(month(tpep_pickup_datetime), levels = 1:12, labels =
month.abb))]
```

```
dt[, `:=`(daypart   = cut(hour, breaks = c(-1, 5, 11, 17, 23),
                          labels = c("Night (0-5)", "Morning (6-11)", "Afternoon
(12-17)", "Evening (18-23)")),
          speed_mph = trip_distance / (duration_min / 60),
          tip_pct   = 100 * tip_amount / fare_amount,
          weekend   = wday(tpep_pickup_datetime) %in% c(1, 7))]

dt <- dt[speed_mph < 100]
dt[, c("tpep_pickup_datetime", "tpep_dropoff_datetime") := NULL]
invisible(gc())

clean_report <- data.table(
  Stage = c("Raw rows", "After removing duplicates", "After validity filters"),
  Rows  = c(n_raw, n_dedup, nrow(dt)))
clean_report[, Retained_pct := round(100 * Rows / n_raw, 2)]
print(clean_report)
cat(sprintf("\nMemory: raw (11 cols) %.0f MB  ->  cleaned (+ features) %.0f MB\n",
            raw_mb, as.numeric(object.size(dt)) / 1024^2))
str(dt)
```

```
                     Stage    Rows Retained_pct
                    <char>   <int>        <num>
1:                  Raw rows 9554778       100.00
2: After removing duplicates 9554777       100.00
3:    After validity filters 8447352        88.41

Memory: raw (11 cols) 656 MB  ->  cleaned (+ features) 902 MB
Classes ‘data.table’ and 'data.frame':  8447352 obs. of  19 variables:
 $ PULocationID   : int  186 140 236 79 211 148 138 161 113 107 ...
 $ DOLocationID   : int  79 236 79 211 148 141 181 261 113 137 ...
 $ passenger_count: int  1 1 1 1 1 1 2 1 1 2 ...
 $ trip_distance  : num  1.72 1.8 4.7 1.4 0.8 ...
 $ payment_type   : Factor w/ 7 levels "Flex fare","Credit card",..: 3 2 2 2 2 2 2
3 3 2 ...
 $ fare_amount    : num  17.7 10 23.3 10 7.9 29.6 45.7 31 3 7.9 ...
 $ tip_amount     : num  0 3.75 3 2 3.2 6.9 10 0 0 0 ...
 $ tolls_amount   : num  0 0 0 0 0 0 0 0 0 0 ...
 $ total_amount   : num  22.7 18.8 31.3 17 16.1 ...
 $ duration_min   : num  19.8 6.6 17.9 8.3 6.1 ...
 $ hour           : int  0 0 0 0 0 0 0 0 0 0 ...
 $ day            : int  1 1 1 1 1 1 1 1 1 1 ...
 $ dow            : Factor w/ 7 levels "Sun","Mon","Tue",..: 2 2 2 2 2 2 2 2 2 2
...
 $ month          : num  1 1 1 1 1 1 1 1 1 1 ...
 $ month_lbl      : Factor w/ 12 levels "Jan","Feb","Mar",..: 1 1 1 1 1 1 1 1 1 1
...
 $ daypart        : Factor w/ 4 levels "Night (0-5)",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ speed_mph      : num  5.21 16.36 15.74 10.12 7.87 ...
 $ tip_pct        : num  0 37.5 12.9 20 40.5 ...
 $ weekend        : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
 - attr(*, ".internal.selfref")=<pointer: 0x59fb4a0163f0>
```

```
zones <- fread(file.path(base_url, "misc/taxi_zone_lookup.csv"))
setnames(zones, c("LocationID", "Borough", "Zone", "service_zone"))
zones[, `:=`(Borough = factor(Borough), Zone = as.character(Zone))]
print(head(zones)); cat("Zones:", nrow(zones), "\n")

t_join <- system.time({
  dt[zones, `:=`(PU_Borough = i.Borough, PU_Zone = i.Zone), on = .(PULocationID =
```

```
LocationID)]
  dt[zones, `:=`(DO_Borough = i.Borough, DO_Zone = i.Zone), on = .(DOLocationID =
LocationID)]
})
cat("Join time for both lookups (s):", t_join["elapsed"], "\n")
cat("Unmatched pickup zones:", dt[is.na(PU_Zone), .N], " | unmatched drop-off
zones:", dt[is.na(DO_Zone), .N], "\n")
print(head(dt[, .(PULocationID, PU_Borough, PU_Zone, DOLocationID, DO_Borough,
DO_Zone, fare_amount, total_amount)]))
```

```
   LocationID       Borough                     Zone service_zone
        <int>        <fctr>                   <char>       <char>
1:          1           EWR          Newark Airport          EWR
2:          2        Queens             Jamaica Bay    Boro Zone
3:          3         Bronx Allerton/Pelham Gardens    Boro Zone
4:          4     Manhattan           Alphabet City  Yellow Zone
5:          5 Staten Island           Arden Heights    Boro Zone
6:          6 Staten Island Arrochar/Fort Wadsworth    Boro Zone
Zones: 265
Join time for both lookups (s): 1.484
Unmatched pickup zones: 0  | unmatched drop-off zones: 0
   PULocationID PU_Borough                      PU_Zone DOLocationID DO_Borough
DO_Zone fare_amount
          <int>     <fctr>                       <char>        <int>     <fctr>
<char>       <num>
1:          186  Manhattan Penn Station/Madison Sq West           79  Manhattan
East Village        17.7
2:          140  Manhattan              Lenox Hill East          236  Manhattan Up
per East Side North        10.0
3:          236  Manhattan        Upper East Side North           79  Manhattan
East Village        23.3
4:           79  Manhattan                East Village          211  Manhattan
SoHo        10.0
5:          211  Manhattan                        SoHo          148  Manhattan
Lower East Side         7.9
6:          148  Manhattan              Lower East Side          141  Manhattan
Lenox Hill West        29.6
   total_amount
          <num>
1:        22.70
2:        18.75
3:        31.30
4:        17.00
5:        16.10
6:        41.50
```

```
trips_hour <- dt[, .(trips = .N, avg_fare = mean(fare_amount), revenue =
sum(total_amount)), by = hour][order(hour)]
print(trips_hour)

trips_dow <- dt[, .(trips = .N, avg_fare = mean(fare_amount), revenue =
sum(total_amount)), by = dow][order(dow)]
print(trips_dow)

trips_month <- dt[, .(trips = .N, avg_fare = mean(fare_amount), revenue =
sum(total_amount),
                      avg_distance = mean(trip_distance)), by = .(month,
month_lbl)][order(month)]
print(trips_month)
```

```
cat(sprintf("\nPeak hour: %02d:00 (%s trips) | quietest hour: %02d:00\n",
            trips_hour[which.max(trips), hour], comma(trips_hour[which.max(trips),
trips]),
            trips_hour[which.min(trips), hour]))
cat("Busiest weekday:", as.character(trips_dow[which.max(trips), dow]),
    "| quietest:", as.character(trips_dow[which.min(trips), dow]), "\n")
cat("Busiest month:", as.character(trips_month[which.max(trips), month_lbl]),
"\n")
```

```
     hour  trips avg_fare  revenue
    <int>  <int>    <num>    <num>
 1:     0 221089 19.29577  6296459
 2:     1 147467 16.93180  3742394
 3:     2  97994 15.99809  2368804
 4:     3  65504 17.21499  1676142
 5:     4  40257 23.17778  1323354
---
20:    19 535258 17.60810 14882639
21:    20 479739 17.97969 13120610
22:    21 477398 18.16613 13109451
23:    22 427032 18.99896 12144025
24:    23 320688 20.23952  9596274
      dow   trips avg_fare  revenue
   <fctr>   <int>    <num>    <num>
1:    Sun 1049144 19.34372 29591695
2:    Mon 1029393 19.63965 29944285
3:    Tue 1167910 18.53601 32415056
4:    Wed 1277617 18.52456 35496059
5:    Thu 1363111 18.81247 38389176
6:    Fri 1277793 18.47899 35389344
7:    Sat 1282384 17.36386 32668212
   month month_lbl   trips avg_fare  revenue avg_distance
   <num>    <fctr>   <int>    <num>    <num>        <num>
1:     1       Jan 2713367 18.38631 74234919     3.296359
2:     2       Feb 2709690 18.33326 74080192     3.224195
3:     3       Mar 3024295 19.10648 85578715     3.392737

Peak hour: 18:00 (609,190 trips) | quietest hour: 04:00
Busiest weekday: Thu | quietest: Mon
Busiest month: Mar
```

```
fare_daypart <- dt[, .(trips = .N, avg_fare = round(mean(fare_amount), 2),
                       avg_total = round(mean(total_amount), 2),
                       revenue = sum(total_amount)), by = daypart][order(daypart)]
print(fare_daypart)

fare_weekend <- dt[, .(trips = .N, avg_fare = round(mean(fare_amount), 2), revenue
= sum(total_amount)),
                   by = .(period = ifelse(weekend, "Weekend", "Weekday"))]
print(fare_weekend)

revenue_daily <- dt[, .(trips = .N, revenue = sum(total_amount), avg_fare =
mean(fare_amount)),
                    by = .(month, day)][order(month, day)]
revenue_daily[, date := as.Date(sprintf("2024-%02d-%02d", month, day))]
print(head(revenue_daily))
```

```
             daypart   trips avg_fare avg_total  revenue
              <fctr>   <int>    <num>     <num>    <num>
1:       Night (0-5)  618068    18.85     27.75 17154348
2:    Morning (6-11) 1851216    18.28     26.30 48686230
3: Afternoon (12-17) 3128763    19.19     28.31 88571531
4:   Evening (18-23) 2849305    18.19     27.90 79481715
    period   trips avg_fare   revenue
    <char>   <int>    <num>     <num>
1: Weekday 6115824    18.77 171633919
2: Weekend 2331528    18.25  62259906
   month   day trips revenue avg_fare       date
   <num> <int> <int>   <num>    <num>     <Date>
1:     1     1 66981 2086311 21.90365 2024-01-01
2:     1     2 70147 2165685 21.27958 2024-01-02
3:     1     3 77114 2253900 20.00092 2024-01-03
4:     1     4 96443 2674531 18.66823 2024-01-04
5:     1     5 96027 2588249 18.05322 2024-01-05
6:     1     6 88261 2263624 17.54302 2024-01-06
```

```
routes <- dt[, .(trips = .N, revenue = sum(total_amount), avg_fare =
mean(fare_amount),
                 avg_dist = mean(trip_distance)), by = .(PU_Zone, DO_Zone)]
routes[, route := paste(PU_Zone, "->", DO_Zone)]

routes_freq <- routes[order(-trips)][1:15]
routes_rev  <- routes[order(-revenue)][1:15]
cat("Top 15 most frequent routes:\n");  print(routes_freq[, .(route, trips,
avg_fare = round(avg_fare, 2))])
cat("\nTop 15 highest-revenue routes:\n"); print(routes_rev[, .(route, trips,
revenue = round(revenue), avg_fare = round(avg_fare, 2))])

top_pu <- dt[, .(trips = .N, revenue = sum(total_amount)), by = PU_Zone][order(-
trips)][1:15]
top_do <- dt[, .(trips = .N, revenue = sum(total_amount)), by = DO_Zone][order(-
trips)][1:15]
cat("\nTop pickup zone:", top_pu$PU_Zone[1], "| top drop-off zone:",
top_do$DO_Zone[1], "\n")
```

```
Top 15 most frequent routes:
                                                    route trips avg_fare
                                                   <char> <int>    <num>
 1:            Upper East Side South -> Upper East Side North 61421     8.76
 2:            Upper East Side North -> Upper East Side South 53928     9.24
 3:            Upper East Side North -> Upper East Side North 42620     6.65
 4:            Upper East Side South -> Upper East Side South 39809     7.20
 5:                   Midtown Center -> Upper East Side South 29791     9.83
 6:              Lincoln Square East -> Upper West Side South 25479     8.05
 7:                   Upper East Side South -> Midtown Center 25377     9.98
 8:                   Midtown Center -> Upper East Side North 25154    13.82
 9:              Upper West Side South -> Lincoln Square East 24985     7.76
10:            Upper West Side South -> Upper West Side North 24075     7.13
11:                     Upper East Side South -> Midtown East 22826     9.35
12:                  Lenox Hill West -> Upper East Side North 21756     8.93
13: Penn Station/Madison Sq West -> Times Sq/Theatre District 20082    12.60
14:                   Upper East Side North -> Midtown Center 20066    14.42
15:            Upper West Side North -> Upper West Side South 19620     7.05

Top 15 highest-revenue routes:
                                           route trips revenue avg_fare
                                          <char> <int>   <num>    <num>
 1:                  JFK Airport -> Outside of NYC 16310 2027029   104.24
 2:       JFK Airport -> Times Sq/Theatre District 18667 1757950    70.33
 3: LaGuardia Airport -> Times Sq/Theatre District 17185 1319740    48.12
 4: Upper East Side South -> Upper East Side North 61421  958771     8.76
 5:                   JFK Airport -> Midtown South 10025  957360    70.26
 6:                    JFK Airport -> Clinton East  9988  933187    70.38
 7: Upper East Side North -> Upper East Side South 53928  864297     9.24
 8: Times Sq/Theatre District -> LaGuardia Airport 11153  846880    49.71
 9:            LaGuardia Airport -> Midtown Center 10344  767791    45.53
10:                  JFK Airport -> Midtown Center  7905  756721    70.27
11:                   JFK Airport -> Midtown North  7826  736720    70.36
12:       Times Sq/Theatre District -> JFK Airport  7953  735858    70.57
13:                     JFK Airport -> Murray Hill  7438  709188    70.15
14:                    JFK Airport -> East Chelsea  6834  653664    70.24
15:            Midtown Center -> LaGuardia Airport  8710  635420    46.61

Top pickup zone: Midtown Center | top drop-off zone: Upper East Side North
```

```
cat("Pearson correlation (distance, fare):", round(cor(dt$trip_distance,
dt$fare_amount), 4), "\n")
fit <- lm(fare_amount ~ trip_distance, data = dt[sample.int(.N, 1e6)])
print(coef(fit))
cat(sprintf("Approx. fare = %.2f + %.2f x miles   (R^2 = %.3f)\n",
            coef(fit)[1], coef(fit)[2], summary(fit)$r.squared))

dist_bins <- dt[, .(trips = .N, avg_fare = mean(fare_amount), avg_per_mile =
mean(fare_amount / trip_distance)),
                by = .(dist_bin = cut(trip_distance, c(0, 1, 2, 3, 5, 10, 20,
100)))][order(dist_bin)]
print(dist_bins)

pay_borough <- dt[, .N, by = .(PU_Borough, payment_type)]
pay_borough[, share := N / sum(N), by = PU_Borough]
print(dcast(pay_borough, PU_Borough ~ payment_type, value.var = "share", fill = 0)
[, lapply(.SD, function(x) if (is.numeric(x)) round(x, 3) else x)])

pay_dist <- dt[, .(trips = .N, avg_fare = mean(fare_amount), avg_tip_pct =
mean(tip_pct), revenue = sum(total_amount)),
```

```
              by = payment_type][order(-trips)]
print(pay_dist)
```

```
Pearson correlation (distance, fare): 0.9548
  (Intercept) trip_distance
     6.348334      3.711942
Approx. fare = 6.35 + 3.71 x miles   (R^2 = 0.912)
   dist_bin   trips  avg_fare avg_per_mile
     <fctr>   <int>     <num>        <num>
1:    (0,1] 2064770  7.579832    12.653319
2:    (1,2] 2837403 11.670070     8.084591
3:    (2,3] 1308078 16.520273     6.785569
4:    (3,5]  888549 22.448308     5.956410
5:   (5,10]  674886 34.552428     4.788427
6:  (10,20]  588641 61.292704     4.148192
7: (20,100]   85025 89.761896     3.680099
Key: <PU_Borough>
      PU_Borough Credit card  Cash No charge Dispute
          <fctr>       <num> <num>     <num>   <num>
1:         Bronx       0.915 0.077     0.003   0.004
2:      Brooklyn       0.883 0.104     0.005   0.008
3:           EWR       0.714 0.200     0.000   0.086
4:     Manhattan       0.844 0.145     0.003   0.008
5:           N/A       0.603 0.286     0.049   0.062
6:        Queens       0.795 0.187     0.005   0.014
7: Staten Island       0.387 0.507     0.067   0.040
8:       Unknown       0.821 0.171     0.003   0.005
   payment_type   trips avg_fare  avg_tip_pct   revenue
         <fctr>   <int>    <num>        <num>     <num>
1:  Credit card 7097532 18.60568 25.340795114 201481403
2:         Cash 1250415 18.66328  0.001351696  29914689
3:      Dispute   70293 20.33642  0.101039980   1817004
4:    No charge   29112 18.17458  0.031393571    680730
```

```
airport <- dt[grepl("Airport", PU_Zone) | grepl("Airport", DO_Zone),
              .(trips = .N, avg_fare = mean(fare_amount), avg_total =
mean(total_amount),
                avg_dist = mean(trip_distance), revenue = sum(total_amount))]
other   <- dt[!(grepl("Airport", PU_Zone) | grepl("Airport", DO_Zone)),
              .(trips = .N, avg_fare = mean(fare_amount), avg_total =
mean(total_amount),
                avg_dist = mean(trip_distance), revenue = sum(total_amount))]
print(rbind(Airport = airport, `Non-airport` = other, idcol = "type"))

speed_hour <- dt[, .(avg_speed = mean(speed_mph)), by = hour][order(hour)]
cat("\nSlowest hour:", speed_hour[which.min(avg_speed), hour], ":00 at",
    round(min(speed_hour$avg_speed), 1), "mph | fastest hour:",
speed_hour[which.max(avg_speed), hour], ":00 at",
    round(max(speed_hour$avg_speed), 1), "mph\n")

print(dt[payment_type %in% c("Credit card", "Cash"), .(avg_tip_pct =
round(mean(tip_pct), 2),
                                                       share_with_tip =
round(mean(tip_amount > 0), 3)),
         by = payment_type])

cat("\nShare of trips that start and end in the same zone:", round(100 * dt[,
mean(PULocationID == DOLocationID)], 2), "%\n")
print(dt[, .(trips = .N, share = round(.N / nrow(dt), 4)), by = passenger_count]
[order(passenger_count)])
```

```
          type   trips avg_fare avg_total  avg_dist   revenue
        <char>   <int>    <num>     <num>     <num>     <num>
1:     Airport  842608 56.22683  77.93260 13.631610  65666633
2: Non-airport 7604744 14.46107  22.12135  2.163825 168227192

Slowest hour: 17 :00 at 9.6 mph | fastest hour: 5 :00 at 19.7 mph
   payment_type avg_tip_pct share_with_tip
         <fctr>       <num>          <num>
1:         Cash        0.00          0.000
2:  Credit card       25.34          0.951

Share of trips that start and end in the same zone: 4.26 %
   passenger_count   trips  share
             <int>   <int>  <num>
1:               1 6627013 0.7845
2:               2 1225775 0.1451
3:               3  277323 0.0328
4:               4  155619 0.0184
5:               5   96770 0.0115
6:               6   64852 0.0077
```

```
fa_base   <- function(d) aggregate(fare_amount ~ hour, data = d, FUN = mean)
fa_vec    <- function(d) rowsum(d$fare_amount, d$hour)[, 1] / tabulate(d$hour +
1L)[unique(sort(d$hour)) + 1L]
fa_tapply <- function(d) tapply(d$fare_amount, d$hour, mean)
fa_lapply <- function(d) unlist(lapply(split(d$fare_amount, d$hour), mean))
fa_purrr  <- function(d) purrr::map_dbl(split(d$fare_amount, d$hour), mean)
fa_dt     <- function(d) d[, .(avg_fare = mean(fare_amount)), by = hour]
[order(hour)]

ref <- fa_dt(dt)$avg_fare
stopifnot(isTRUE(all.equal(as.numeric(fa_vec(dt)),    ref)),
          isTRUE(all.equal(as.numeric(fa_tapply(dt)), ref)),
          isTRUE(all.equal(as.numeric(fa_lapply(dt)), ref)),
          isTRUE(all.equal(as.numeric(fa_purrr(dt)),  ref)),
          isTRUE(all.equal(as.numeric(fa_base(dt)$fare_amount), ref)))
cat("All 6 implementations of Operation A agree.\n")
```

```
All 6 implementations of Operation A agree.
```

```
smp <- as.data.frame(dt[sample.int(.N, 1e5), .(fare_amount, tip_amount,
tolls_amount, total_amount)])

tip_apply  <- function(d) apply(as.matrix(d), 1, function(r)
unname(r["tip_amount"] / r["fare_amount"] * 100))
tip_lapply <- function(d) unlist(lapply(seq_len(nrow(d)), function(i)
d$tip_amount[i] / d$fare_amount[i] * 100))
tip_purrr  <- function(d) purrr::map2_dbl(d$tip_amount, d$fare_amount, function(t,
f) t / f * 100)
tip_vec    <- function(d) d$tip_amount / d$fare_amount * 100
tip_dt     <- function(d) { x <- as.data.table(d); x[, tip_pct := tip_amount /
fare_amount * 100]; x$tip_pct }

stopifnot(isTRUE(all.equal(tip_apply(smp),  tip_vec(smp))),
          isTRUE(all.equal(tip_lapply(smp), tip_vec(smp))),
          isTRUE(all.equal(tip_purrr(smp),  tip_vec(smp))),
          isTRUE(all.equal(tip_dt(smp),     tip_vec(smp))))
cat("All 5 implementations of Operation B agree.\n")
```

```
All 5 implementations of Operation B agree.
```

```
approaches_A <- list(`base aggregate()` = fa_base, `vectorized rowsum()` = fa_vec,
`vectorized tapply()` = fa_tapply,
                     `functional lapply(split)` = fa_lapply, `functional
purrr::map` = fa_purrr, `data.table` = fa_dt)
approaches_B <- list(`functional apply()` = tip_apply, `functional lapply()` =
tip_lapply,
                     `functional purrr::map2` = tip_purrr, `vectorized` = tip_vec,
`data.table :=` = tip_dt)

mem_A <- rbindlist(lapply(names(approaches_A), function(n) {
  m <- mem_time(approaches_A[[n]](dt)); data.table(Operation = "A: fare by hour
(full data)", Approach = n,
                                                   Elapsed_s = round(m$elapsed,
3), Peak_mem_MB = round(m$peak_mb))
}))
mem_B <- rbindlist(lapply(names(approaches_B), function(n) {
  m <- mem_time(approaches_B[[n]](smp)); data.table(Operation = "B: tip % (100k
rows)", Approach = n,
                                                    Elapsed_s = round(m$elapsed,
3), Peak_mem_MB = round(m$peak_mb))
}))
mem_tab <- rbind(mem_A, mem_B)
print(mem_tab)
```

```
                     Operation                Approach Elapsed_s Peak_mem_MB
                        <char>                  <char>     <num>       <num>
 1: A: fare by hour (full data)         base aggregate()     3.377        2857
 2: A: fare by hour (full data)      vectorized rowsum()     0.350        1669
 3: A: fare by hour (full data)      vectorized tapply()     0.379        1700
 4: A: fare by hour (full data) functional lapply(split)     0.284        1636
 5: A: fare by hour (full data)    functional purrr::map     0.278        1636
 6: A: fare by hour (full data)               data.table     0.262        1584
 7:        B: tip % (100k rows)       functional apply()     0.489        1531
 8:        B: tip % (100k rows)      functional lapply()     0.282        1507
 9:        B: tip % (100k rows)   functional purrr::map2     0.284        1430
10:        B: tip % (100k rows)               vectorized     0.001        1346
11:        B: tip % (100k rows)            data.table :=     0.011        1351
```

```
parts <- split(dt[, .(month, trip_distance, fare_amount)], by = "month", keep.by =
TRUE)
cat("Partitions:", paste(names(parts), collapse = ", "), "| rows:",
paste(sapply(parts, nrow), collapse = ", "), "\n")

heavy_fn <- function(d, B = 300, m = 100000) {
  set.seed(1)
  x <- d$trip_distance; y <- d$fare_amount; n <- length(x)
  slopes <- numeric(B)
  for (b in seq_len(B)) {
    i <- sample.int(n, m, replace = TRUE)
    xs <- x[i]; ys <- y[i]
    slopes[b] <- cov(xs, ys) / var(xs)
  }
  data.table::data.table(month = d$month[1], n_trips = n, slope_mean =
mean(slopes), slope_sd = sd(slopes))
}
```

```
Partitions: 1, 2, 3 | rows: 2713367, 2709690, 3024295
```

```
res_seq <- rbindlist(lapply(parts, heavy_fn))
t_seq_runs <- replicate(3, system.time(rbindlist(lapply(parts, heavy_fn))))
```

```
["elapsed"])
t_seq <- median(t_seq_runs)
cat("Sequential times (3 runs):", round(t_seq_runs, 2), "-> median", round(t_seq,
2), "s\n")
print(res_seq)
```

```
Sequential times (3 runs): 10.42 10.42 9.61 -> median 10.42 s
   month n_trips slope_mean   slope_sd
   <num>   <int>      <num>      <num>
1:     1 2713367   3.705596 0.01653577
2:     2 2709690   3.714708 0.01552625
3:     3 3024295   3.712917 0.01553833
```

```
n_cores <- max(1, min(parallel::detectCores(), length(parts)))
cat("Using", n_cores, "worker(s) for", length(parts), "partitions\n")

run_parallel <- function(cores) {
  cl <- parallel::makeCluster(cores)
  doParallel::registerDoParallel(cl)
  on.exit({ parallel::stopCluster(cl); foreach::registerDoSEQ() })
  system.time(res <- foreach(d = parts, .combine = rbind, .packages =
"data.table",
                             .export = "heavy_fn") %dopar% heavy_fn(d))["elapsed"]
-> el
  list(elapsed = unname(el), result = res)
}

par_runs <- replicate(3, run_parallel(n_cores)$elapsed)
t_par <- median(par_runs)
res_par <- run_parallel(n_cores)$result
cat("Parallel times (3 runs):", round(par_runs, 2), "-> median", round(t_par, 2),
"s\n")
cat("Results identical to sequential:", isTRUE(all.equal(as.data.frame(res_par),
as.data.frame(res_seq))), "\n")

speedup    <- t_seq / t_par
efficiency <- speedup / n_cores
cat(sprintf("\nSpeedup = %.2f / %.2f = %.2fx  | parallel efficiency = %.0f%%
(ideal speedup = min(cores, partitions) = %d)\n",
            t_seq, t_par, speedup, 100 * efficiency, n_cores))
```

```
Using 2 worker(s) for 3 partitions
Parallel times (3 runs): 10.17 12.75 11.65 -> median 11.65 s
Results identical to sequential: TRUE

Speedup = 10.42 / 11.65 = 0.89x  | parallel efficiency = 45% (ideal speedup = min
(cores, partitions) = 2)
```

```
hour_parts <- split(dt[, .(hour, fare_amount)], by = "hour")
light_fn   <- function(d) data.table::data.table(hour = d$hour[1], avg =
mean(d$fare_amount))

t_seq_light <- median(replicate(5, system.time(rbindlist(lapply(hour_parts,
light_fn)))["elapsed"]))
cl <- parallel::makeCluster(n_cores); doParallel::registerDoParallel(cl)
t_par_light <- median(replicate(5, system.time(
  foreach(d = hour_parts, .combine = rbind, .packages = "data.table", .export =
"light_fn") %dopar% light_fn(d)
)["elapsed"]))
parallel::stopCluster(cl); foreach::registerDoSEQ()
```

```
cat(sprintf("Light task  | sequential: %.3f s | parallel: %.3f s | speedup:
%.2fx\n",
            t_seq_light, t_par_light, t_seq_light / t_par_light))
```

```
Light task  | sequential: 0.027 s | parallel: 0.823 s | speedup: 0.03x
```

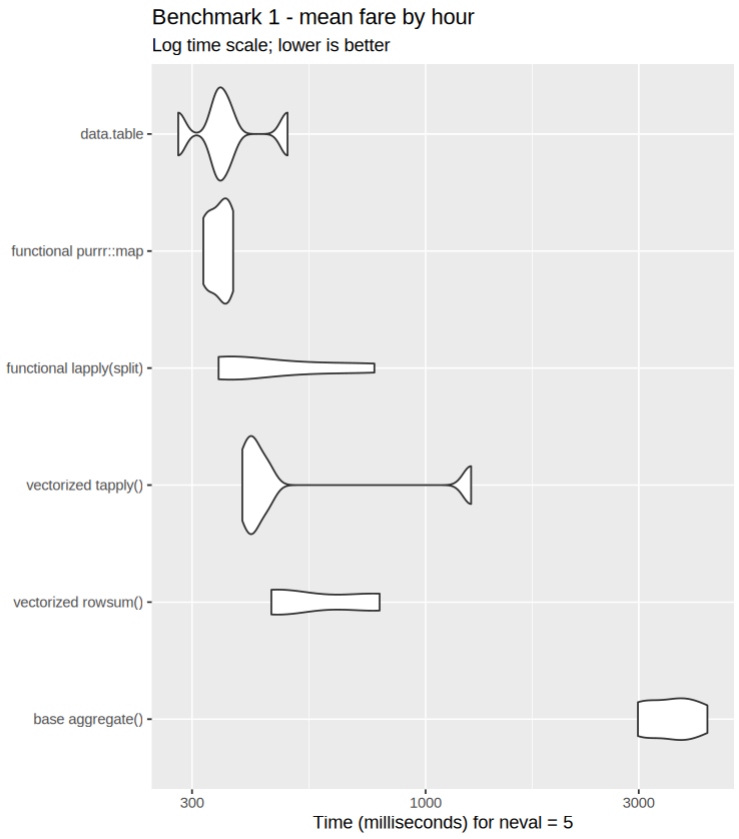
```
core_set <- sort(unique(c(1, seq_len(n_cores))))
scal <- rbindlist(lapply(core_set, function(k) {
  t <- if (k == 1) t_seq else median(replicate(2, run_parallel(k)$elapsed))
  data.table(Workers = k, Time_s = round(t, 2), Speedup = round(t_seq / t, 2))
}))
print(scal)
```

```
   Workers Time_s Speedup
     <num>  <num>   <num>
1:       1  10.42       1
2:       2  10.44       1
```

```
mb1 <- microbenchmark(
  `base aggregate()`         = fa_base(dt),
  `vectorized rowsum()`      = fa_vec(dt),
  `vectorized tapply()`      = fa_tapply(dt),
  `functional lapply(split)` = fa_lapply(dt),
  `functional purrr::map`    = fa_purrr(dt),
  `data.table`               = fa_dt(dt),
  times = 5, unit = "ms")
print(mb1)
tab1 <- mb_tab(mb1, "A: mean fare by hour (full data)")
autoplot(mb1) + labs(title = "Benchmark 1 - mean fare by hour", subtitle = "Log
time scale; lower is better")
```

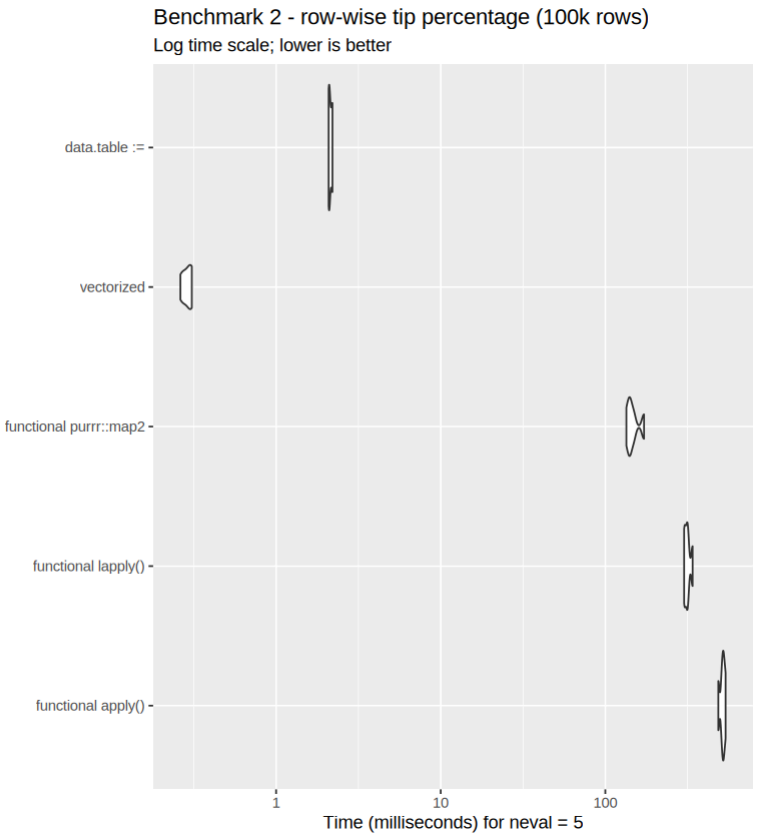
```
Unit: milliseconds
                    expr       min        lq      mean    median        uq
max neval
         base aggregate() 2987.2148 3107.7200 3561.0541 3663.1449 3772.7992 4274.3
918     5
      vectorized rowsum()  451.5294  460.4277  589.1945  474.7687  770.3354  788.9
111     5
      vectorized tapply()  388.7664  406.7557  583.0453  413.0556  442.3951 1264.2
539     5
 functional lapply(split)  343.9809  346.1100  473.9190  381.2760  530.3411  767.8
871     5
    functional purrr::map  317.9740  330.4951  345.5277  350.1970  358.1759  370.7
963     5
              data.table  279.3827  338.2335  364.0254  347.3463  364.3319  490.8
326     5
```

```
Warning message:
“`aes_string()` was deprecated in ggplot2 3.0.0.
ℹ Please use tidy evaluation idioms with `aes()`.
ℹ See also `vignette("ggplot2-in-packages")` for more information.
ℹ The deprecated feature was likely used in the microbenchmark package.
  Please report the issue at <https://github.com/joshuaulrich/microbenchmark/issue
s/>.”
```




```
mb2 <- microbenchmark(
  `functional apply()`     = tip_apply(smp),
  `functional lapply()`    = tip_lapply(smp),
  `functional purrr::map2`= tip_purrr(smp),
  `vectorized`             = tip_vec(smp),
  `data.table :=`          = tip_dt(smp),
  times = 5, unit = "ms")
print(mb2)
tab2 <- mb_tab(mb2, "B: tip % per trip (100k rows)")
autoplot(mb2) + labs(title = "Benchmark 2 - row-wise tip percentage (100k rows)",
subtitle = "Log time scale; lower is better")
```

```
Unit: milliseconds
                  expr        min         lq        mean     median         uq
max neval
     functional apply() 486.025618 509.975749 515.8016916 519.158514 525.656435 53
8.192142     5
    functional lapply() 301.252733 304.878136 315.5389426 315.744412 316.494963 33
9.324469     5
 functional purrr::map2 134.309107 140.114985 147.3733442 141.644616 148.912791 17
1.885222     5
             vectorized   0.262141   0.276171   0.2889556   0.291918   0.307085
0.307463     5
          data.table :=   2.084996   2.102079   2.1381000   2.106447   2.197972
2.199006     5
```




```
df <- as.data.frame(dt[, .(PU_Borough, payment_type, fare_amount, total_amount,
trip_distance)])

base_pipeline <- function(d) {
  s <- d[d$trip_distance > 2 & d$fare_amount > 10, ]
  r <- aggregate(cbind(total_amount, fare_amount) ~ PU_Borough + payment_type,
data = s, FUN = mean)
  r[order(-r$total_amount), ]
}
dt_pipeline <- function(d) d[trip_distance > 2 & fare_amount > 10,
                             .(total_amount = mean(total_amount), fare_amount =
mean(fare_amount)),
                             by = .(PU_Borough, payment_type)][order(-
total_amount)]
dtsub <- dt[, .(PU_Borough, payment_type, fare_amount, total_amount,
trip_distance)]

mb3 <- microbenchmark(`base R pipeline` = base_pipeline(df), `data.table pipeline`
= dt_pipeline(dtsub),
                      times = 3, unit = "ms")
print(mb3)
tab3 <- mb_tab(mb3, "C: filter + group pipeline (full data)")
rm(df); invisible(gc())
```

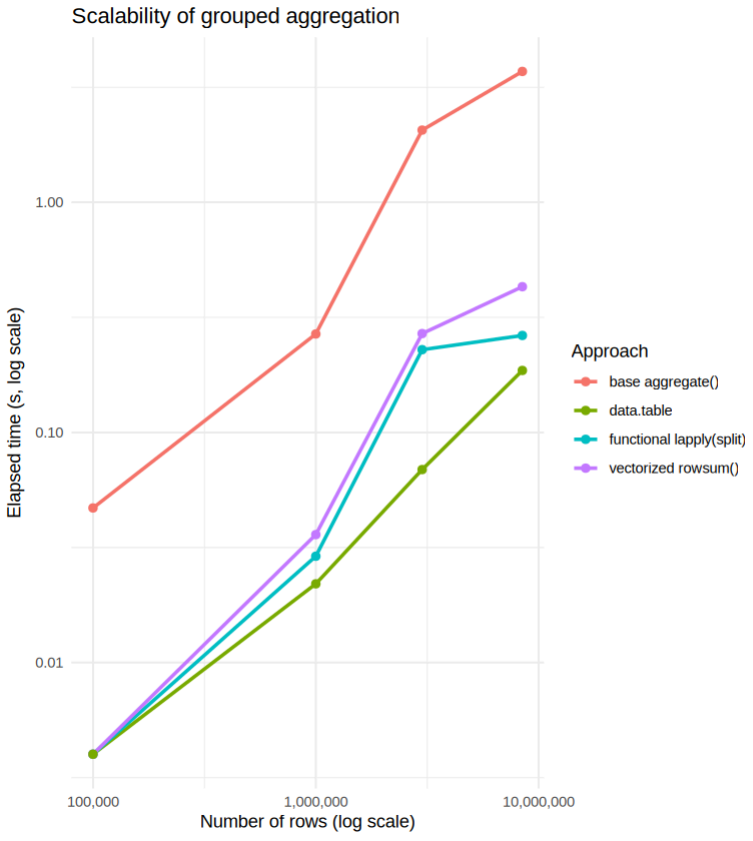
```
Unit: milliseconds
                expr       min        lq      mean    median        uq       max n
eval
     base R pipeline 2235.5395 2352.2642 3007.8335 2468.9889 3393.9804 4318.9720
3
 data.table pipeline  354.0447  375.2275  451.7087  396.4103  500.5406  604.6709
3
```

```
sizes <- c(1e5, 1e6, 3e6, nrow(dt))
sizes <- sort(unique(pmin(sizes, nrow(dt))))
scale_res <- rbindlist(lapply(sizes, function(n) {
```

```
  d <- dt[sample.int(.N, n)]
  rbindlist(list(
    data.table(Rows = n, Approach = "base aggregate()",        Time_s =
system.time(fa_base(d))["elapsed"]),
    data.table(Rows = n, Approach = "functional lapply(split)",Time_s =
system.time(fa_lapply(d))["elapsed"]),
    data.table(Rows = n, Approach = "vectorized rowsum()",     Time_s =
system.time(fa_vec(d))["elapsed"]),
    data.table(Rows = n, Approach = "data.table",              Time_s =
system.time(fa_dt(d))["elapsed"])))
}))
print(dcast(scale_res, Rows ~ Approach, value.var = "Time_s"))

ggplot(scale_res, aes(Rows, Time_s, colour = Approach)) +
  geom_line(linewidth = 1) + geom_point(size = 2) +
  scale_x_log10(labels = comma) + scale_y_log10() +
  labs(title = "Scalability of grouped aggregation", x = "Number of rows (log
scale)",
       y = "Elapsed time (s, log scale)", colour = "Approach") + theme_minimal()
```

```
Key: <Rows>
      Rows base aggregate() data.table functional lapply(split) vectorized rowsum
()
     <num>            <num>      <num>                    <num>                <num
>
1:  100000            0.047      0.004                    0.004                0.00
4
2: 1000000            0.268      0.022                    0.029                0.03
6
3: 3000000            2.060      0.069                    0.229                0.26
9
4: 8447352            3.705      0.186                    0.264                0.43
0
```




```
seq_par <- data.table(
  Benchmark = "D: bootstrap per month (3 partitions)",
```

```
  Approach  = c("sequential lapply", sprintf("parallel foreach/doParallel (%d
workers)", n_cores)),
  Min_ms = NA_real_, Median_ms = round(1000 * c(t_seq, t_par), 2), Mean_ms =
NA_real_, Max_ms = NA_real_)
seq_par_light <- data.table(
  Benchmark = "E: light task (24 partitions)",
  Approach  = c("sequential lapply", sprintf("parallel foreach/doParallel (%d
workers)", n_cores)),
  Min_ms = NA_real_, Median_ms = round(1000 * c(t_seq_light, t_par_light), 2),
Mean_ms = NA_real_, Max_ms = NA_real_)

results <- rbindlist(list(tab1, tab2, tab3, seq_par, seq_par_light))
results[, Relative_to_fastest := round(Median_ms / min(Median_ms), 1), by =
Benchmark]
print(results[, .(Benchmark, Approach, Median_ms, Relative_to_fastest)], nrows =
100)

cat("\nPeak memory (MB) during each implementation:\n"); print(mem_tab)
cat(sprintf("\nSpeedup (sequential/parallel) - heavy task: %.2fx | light task:
%.2fx\n",
            t_seq / t_par, t_seq_light / t_par_light))
```

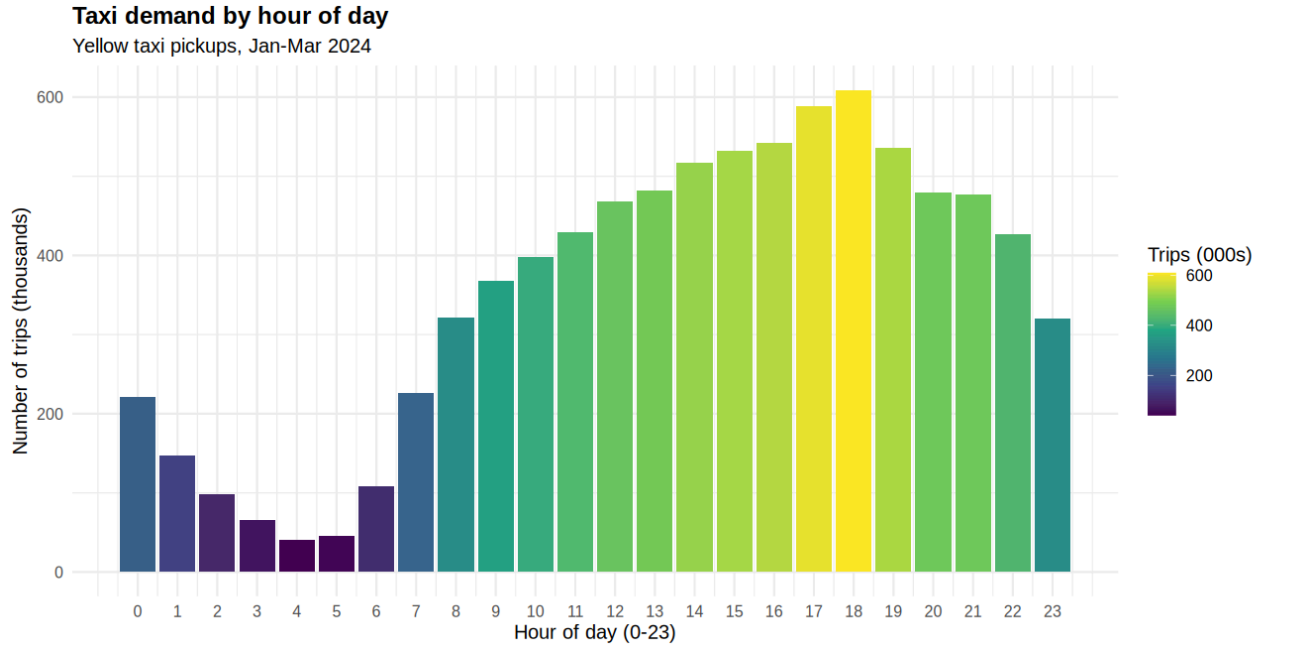
```
                                  Benchmark                                  Approach
Median_ms Relative_to_fastest
                                     <char>                                     <char>
<num>               <num>
 1:       A: mean fare by hour (full data)                       base aggregate()
3663.14                10.5
 2:       A: mean fare by hour (full data)                     vectorized rowsum()
474.77                 1.4
 3:       A: mean fare by hour (full data)                     vectorized tapply()
413.06                 1.2
 4:       A: mean fare by hour (full data)                functional lapply(split)
381.28                 1.1
 5:       A: mean fare by hour (full data)                   functional purrr::map
350.20                 1.0
 6:       A: mean fare by hour (full data)                              data.table
347.35                 1.0
 7:          B: tip % per trip (100k rows)                     functional apply()
519.16              1790.2
 8:          B: tip % per trip (100k rows)                     functional lapply()
315.74              1088.8
 9:          B: tip % per trip (100k rows)                  functional purrr::map2
141.64               488.4
10:          B: tip % per trip (100k rows)                              vectorized
0.29                 1.0
11:          B: tip % per trip (100k rows)                           data.table :=
2.11                 7.3
12: C: filter + group pipeline (full data)                          base R pipeline
2468.99                6.2
13: C: filter + group pipeline (full data)                     data.table pipeline
396.41                 1.0
14:  D: bootstrap per month (3 partitions)                       sequential lapply
10419.00                 1.0
15:  D: bootstrap per month (3 partitions) parallel foreach/doParallel (2 workers)
11649.00                 1.1
16:          E: light task (24 partitions)                       sequential lapply
27.00                  1.0
17:          E: light task (24 partitions) parallel foreach/doParallel (2 workers)
823.00                 30.5

Peak memory (MB) during each implementation:
                     Operation                 Approach Elapsed_s Peak_mem_MB
                        <char>                   <char>     <num>       <num>
 1: A: fare by hour (full data)         base aggregate()     3.377        2857
 2: A: fare by hour (full data)      vectorized rowsum()     0.350        1669
 3: A: fare by hour (full data)      vectorized tapply()     0.379        1700
 4: A: fare by hour (full data) functional lapply(split)     0.284        1636
 5: A: fare by hour (full data)    functional purrr::map     0.278        1636
 6: A: fare by hour (full data)               data.table     0.262        1584
 7:        B: tip % (100k rows)       functional apply()     0.489        1531
 8:        B: tip % (100k rows)      functional lapply()     0.282        1507
 9:        B: tip % (100k rows)   functional purrr::map2     0.284        1430
10:        B: tip % (100k rows)               vectorized     0.001        1346
11:        B: tip % (100k rows)            data.table :=     0.011        1351

Speedup (sequential/parallel) - heavy task: 0.89x | light task: 0.03x
```

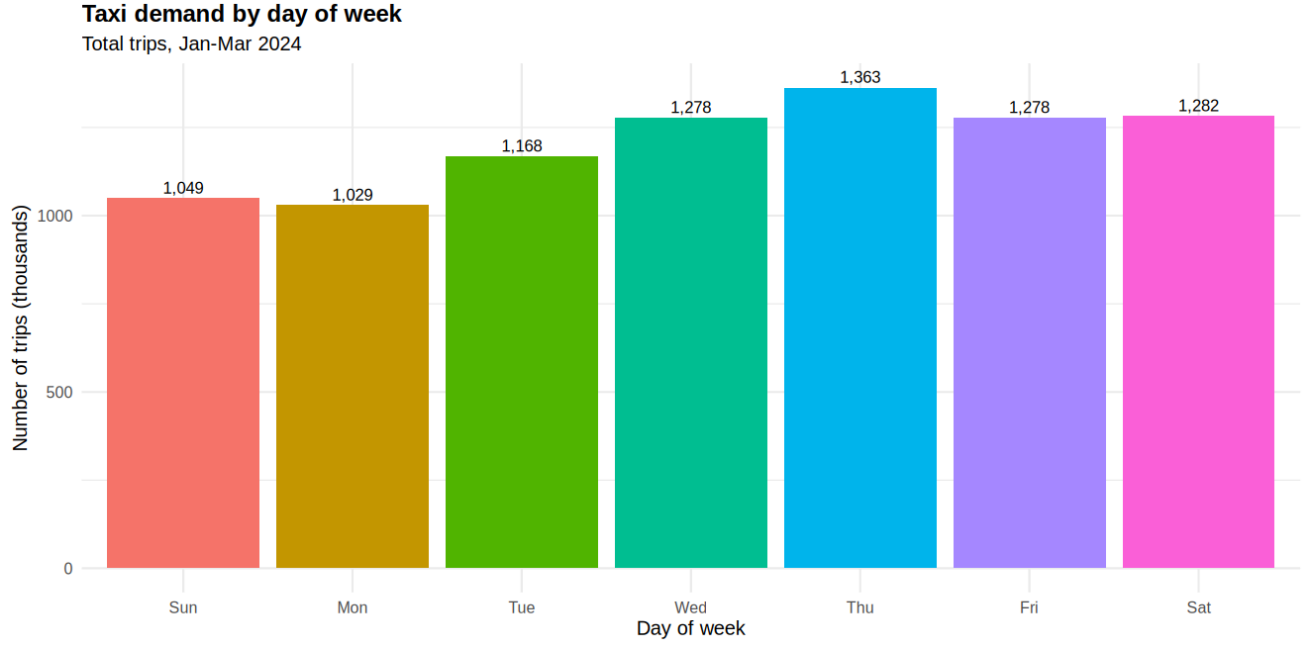
```
theme_set(theme_minimal(base_size = 12) + theme(plot.title = element_text(face =
"bold")))
options(repr.plot.width = 11, repr.plot.height = 5.5)
```

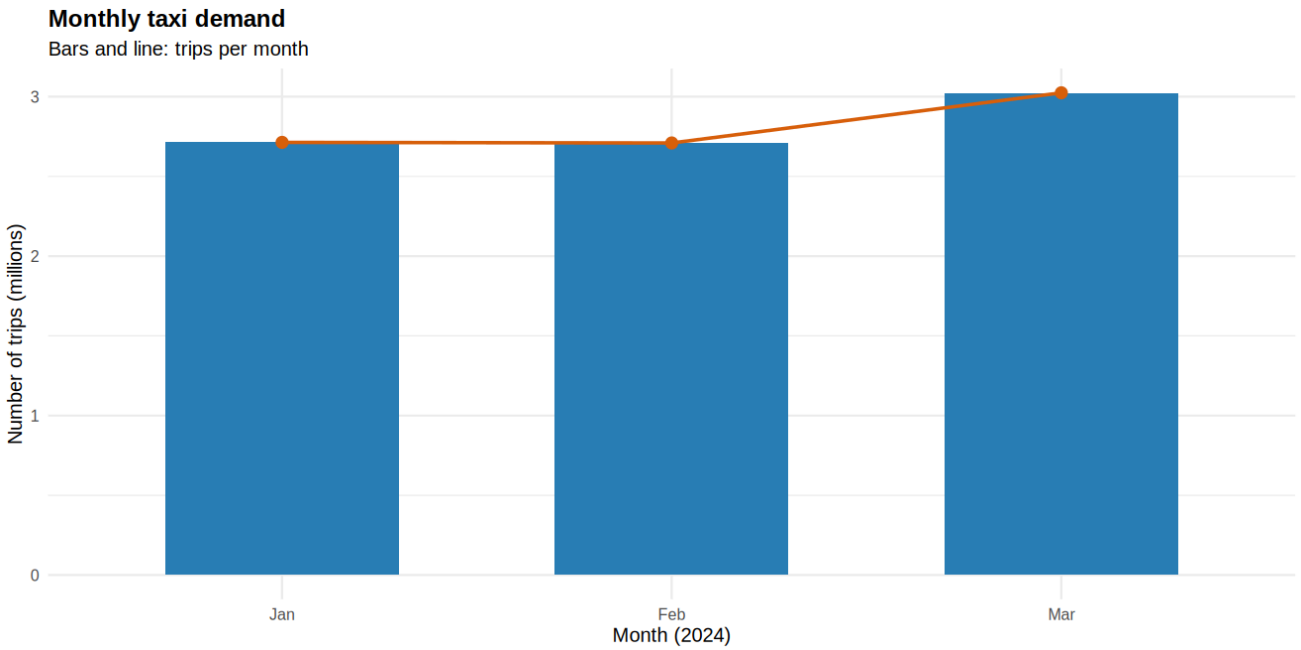
```
ggplot(trips_hour, aes(hour, trips / 1e3)) +
  geom_col(aes(fill = trips / 1e3)) + scale_fill_viridis_c(name = "Trips (000s)")
+
  scale_x_continuous(breaks = 0:23) +
  labs(title = "Taxi demand by hour of day", subtitle = "Yellow taxi pickups, Jan-
Mar 2024",
       x = "Hour of day (0-23)", y = "Number of trips (thousands)")
```




```
ggplot(trips_dow, aes(dow, trips / 1e3, fill = dow)) + geom_col(show.legend =
FALSE) +
  geom_text(aes(label = comma(round(trips / 1e3))), vjust = -0.4, size = 3.5) +
  labs(title = "Taxi demand by day of week", subtitle = "Total trips, Jan-Mar
2024",
       x = "Day of week", y = "Number of trips (thousands)")

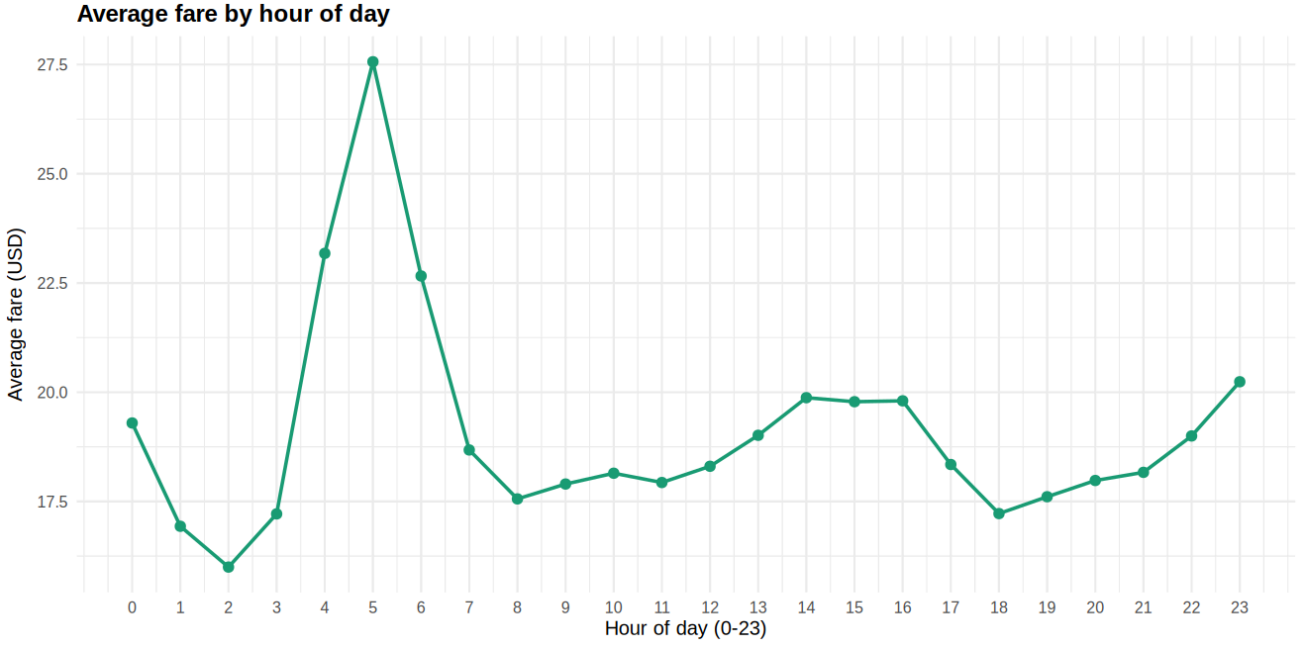
ggplot(trips_month, aes(month_lbl, trips / 1e6, group = 1)) +
  geom_col(fill = "#2c7fb8", width = 0.6) + geom_line(colour = "#d95f0e",
linewidth = 1) + geom_point(colour = "#d95f0e", size = 3) +
  labs(title = "Monthly taxi demand", subtitle = "Bars and line: trips per month",
       x = "Month (2024)", y = "Number of trips (millions)")
```

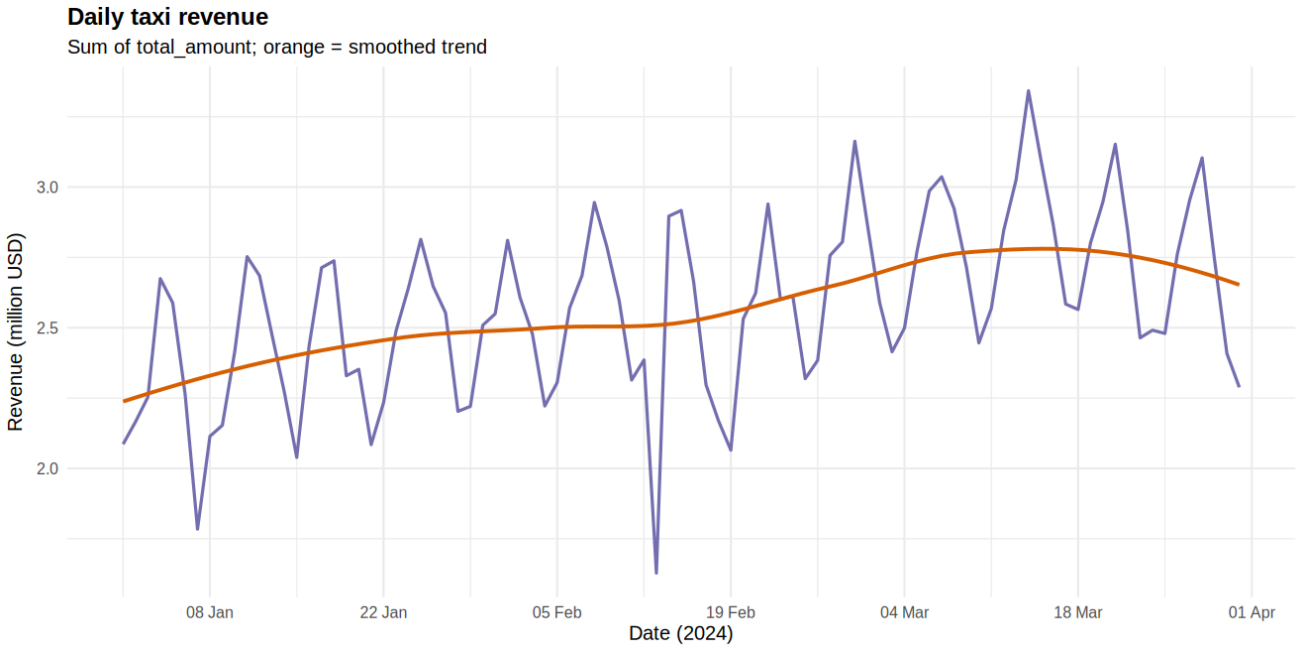




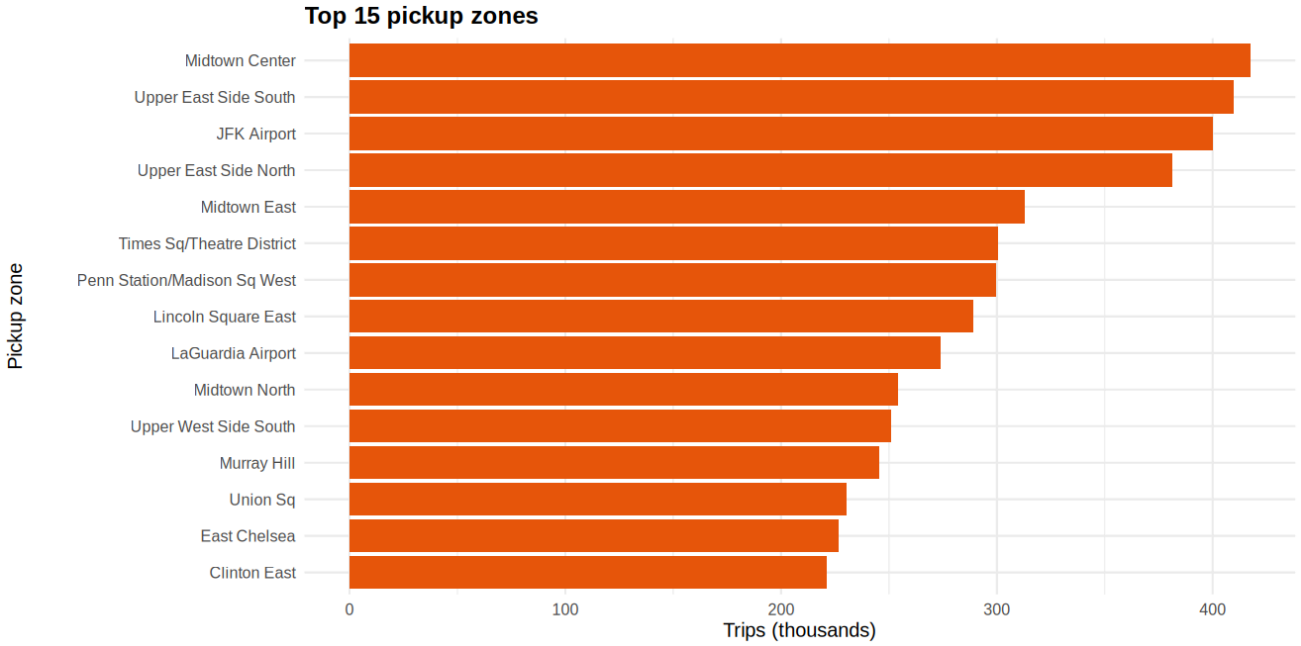

```
ggplot(trips_hour, aes(hour, avg_fare)) + geom_line(linewidth = 1, colour =
"#1b9e77") + geom_point(size = 2.5, colour = "#1b9e77") +
  scale_x_continuous(breaks = 0:23) +
  labs(title = "Average fare by hour of day", x = "Hour of day (0-23)", y =
"Average fare (USD)")

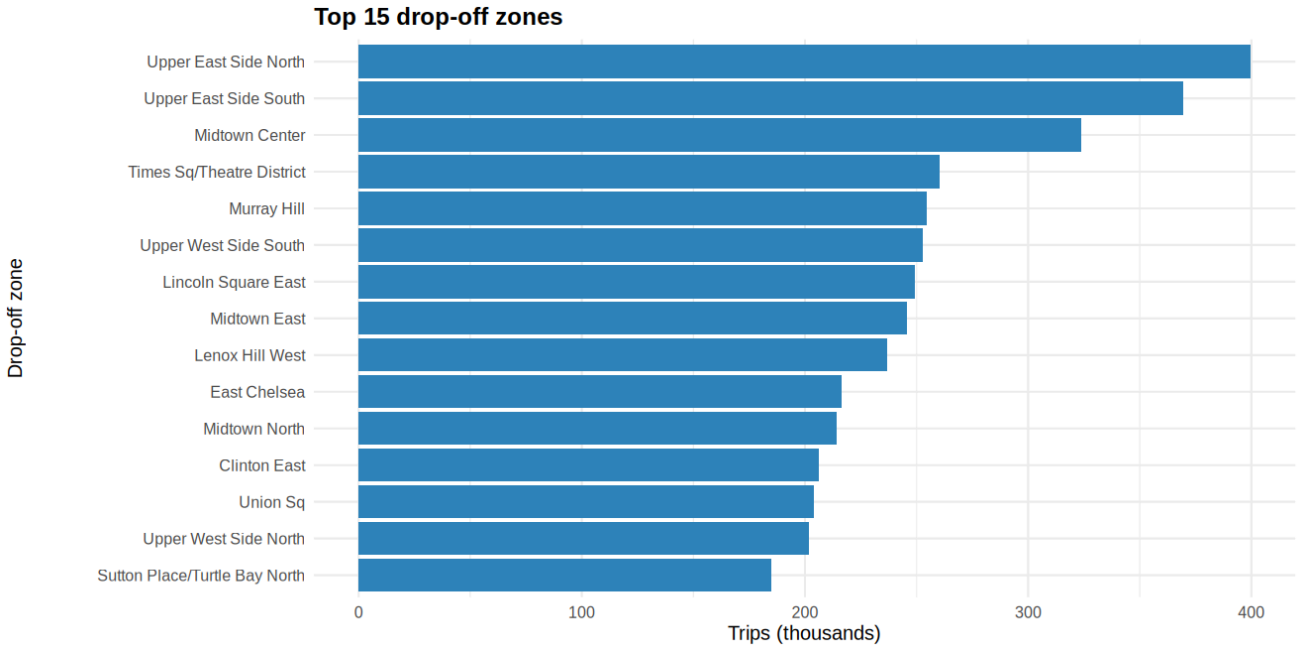
ggplot(revenue_daily, aes(date, revenue / 1e6)) +
  geom_line(colour = "#7570b3", linewidth = 0.9) + geom_smooth(method = "loess",
formula = y ~ x, se = FALSE, colour = "#d95f02") +
  scale_x_date(date_breaks = "2 weeks", date_labels = "%d %b") +
  labs(title = "Daily taxi revenue", subtitle = "Sum of total_amount; orange =
smoothed trend",
       x = "Date (2024)", y = "Revenue (million USD)")
```



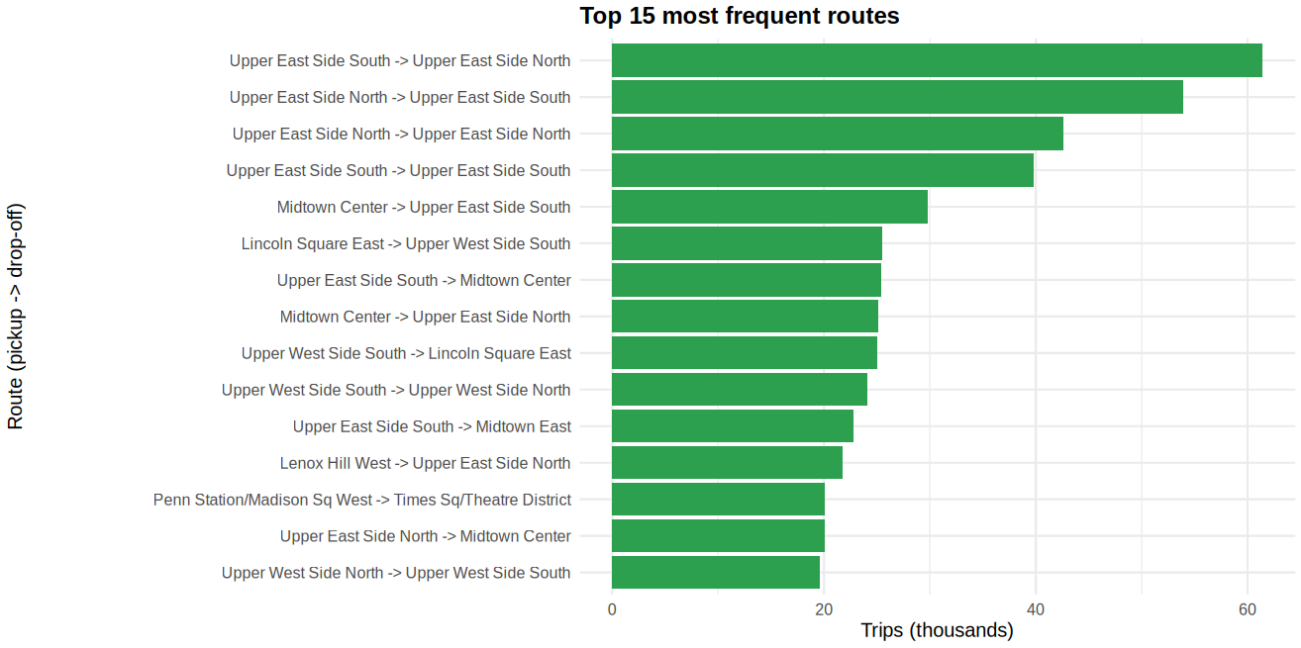


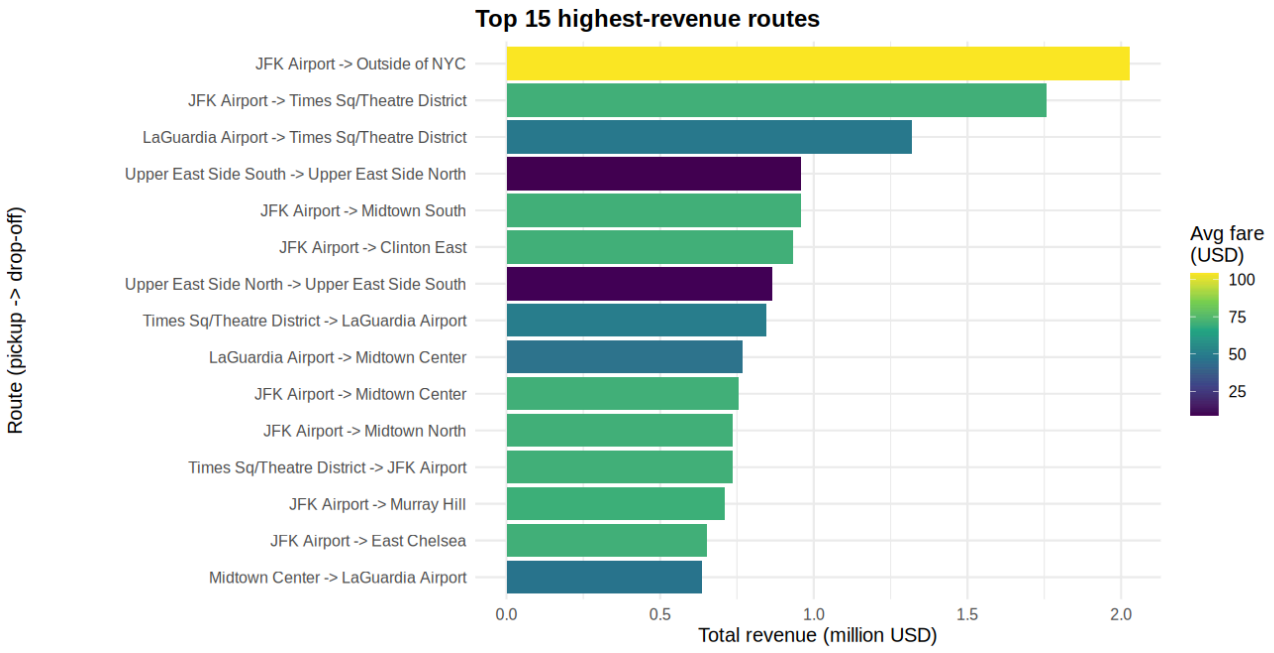

```
ggplot(top_pu, aes(reorder(PU_Zone, trips), trips / 1e3)) + geom_col(fill =
"#e6550d") + coord_flip() +
  labs(title = "Top 15 pickup zones", x = "Pickup zone", y = "Trips (thousands)")
ggplot(top_do, aes(reorder(DO_Zone, trips), trips / 1e3)) + geom_col(fill =
"#3182bd") + coord_flip() +
  labs(title = "Top 15 drop-off zones", x = "Drop-off zone", y = "Trips
(thousands)")
```





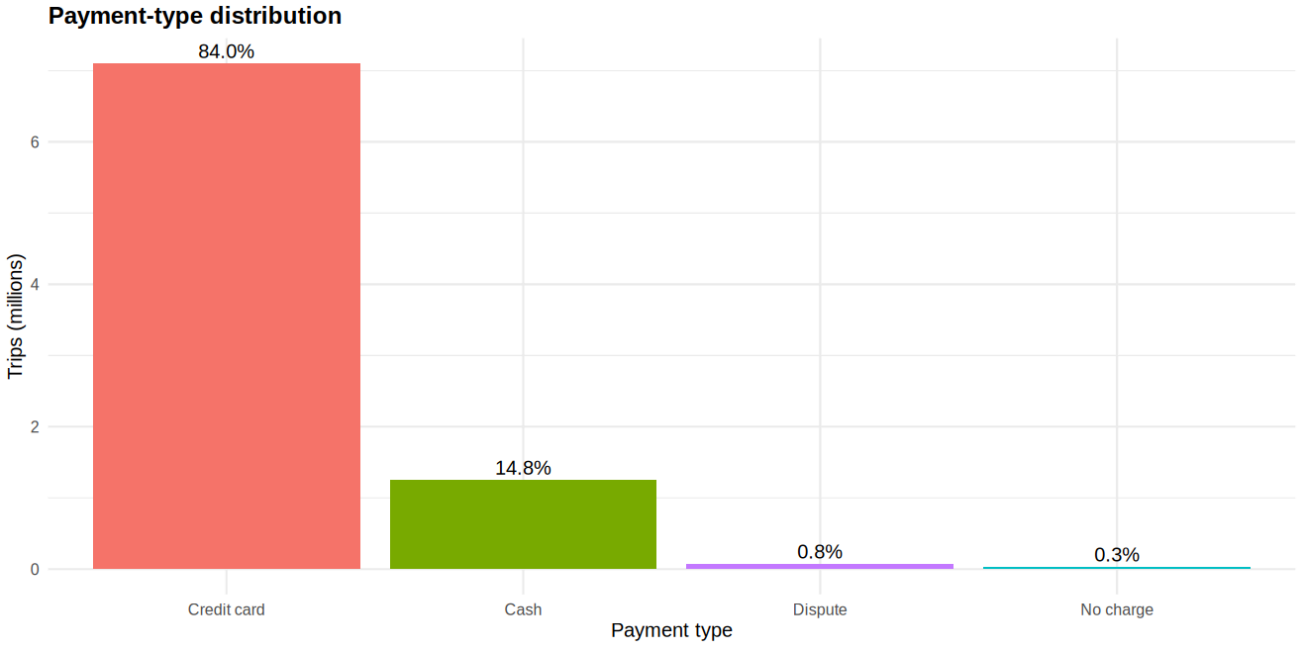

```
ggplot(routes_freq, aes(reorder(route, trips), trips / 1e3)) + geom_col(fill =
"#31a354") + coord_flip() +
  labs(title = "Top 15 most frequent routes", x = "Route (pickup -> drop-off)", y
= "Trips (thousands)")
ggplot(routes_rev, aes(reorder(route, revenue), revenue / 1e6, fill = avg_fare)) +
geom_col() + coord_flip() +
  scale_fill_viridis_c(name = "Avg fare\n(USD)") +
  labs(title = "Top 15 highest-revenue routes", x = "Route (pickup -> drop-off)",
y = "Total revenue (million USD)")
```

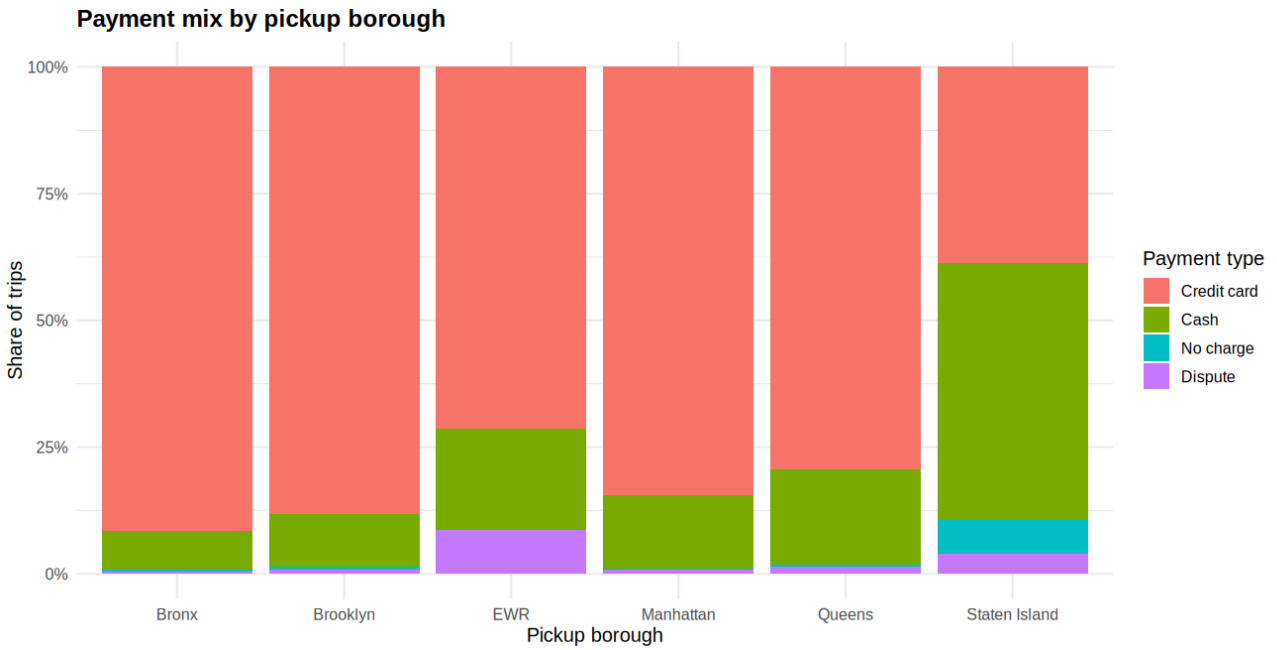





```
ggplot(pay_dist, aes(reorder(payment_type, -trips), trips / 1e6, fill =
payment_type)) + geom_col(show.legend = FALSE) +
  geom_text(aes(label = percent(trips / sum(trips), accuracy = 0.1)), vjust =
-0.4) +
  labs(title = "Payment-type distribution", x = "Payment type", y = "Trips
(millions)")

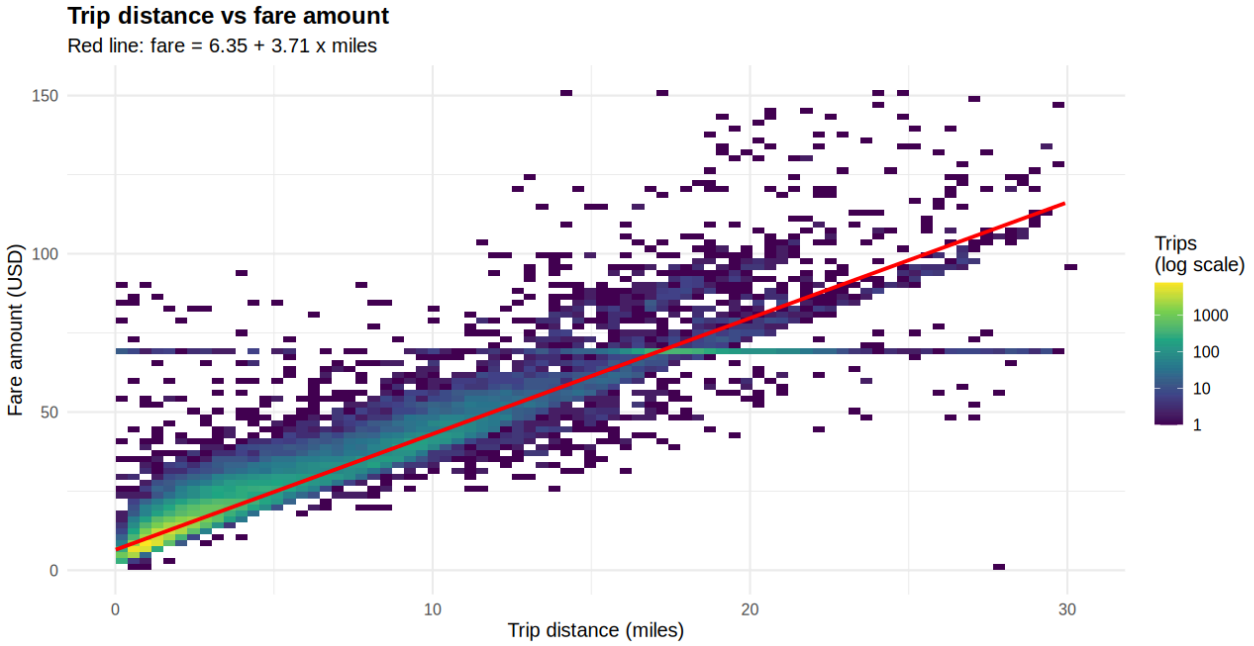
pb <- pay_borough[PU_Borough %in% c("Manhattan", "Queens", "Brooklyn", "Bronx",
"Staten Island", "EWR")]
ggplot(pb, aes(PU_Borough, share, fill = payment_type)) + geom_col(position =
"fill") +
  scale_y_continuous(labels = percent) + labs(title = "Payment mix by pickup
borough", x = "Pickup borough",
                                              y = "Share of trips", fill =
"Payment type")
```

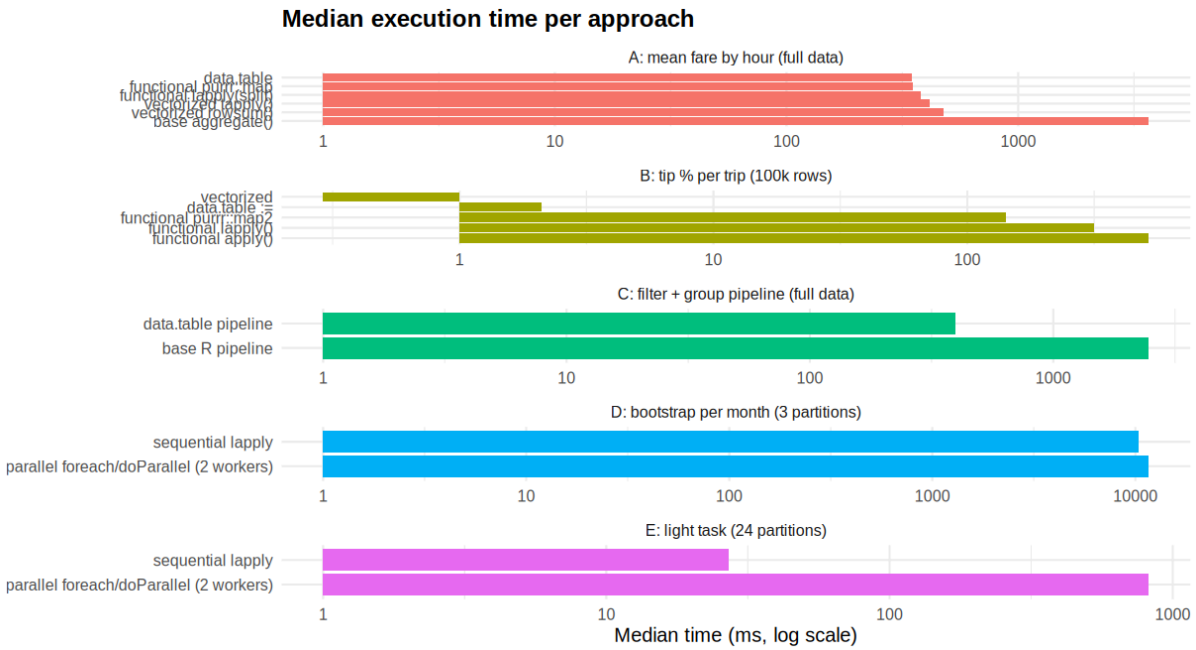





```
set.seed(7)
smp_plot <- dt[sample.int(.N, 1e5)][trip_distance <= 30 & fare_amount <= 150]
ggplot(smp_plot, aes(trip_distance, fare_amount)) +
  geom_bin2d(bins = 80) + scale_fill_viridis_c(trans = "log10", name =
"Trips\n(log scale)") +
  geom_smooth(method = "lm", formula = y ~ x, colour = "red", se = FALSE) +
  labs(title = "Trip distance vs fare amount", subtitle = sprintf("Red line: fare
= %.2f + %.2f x miles", coef(fit)[1], coef(fit)[2]),
       x = "Trip distance (miles)", y = "Fare amount (USD)")

ggplot(results[Median_ms > 0], aes(reorder(Approach, -Median_ms), Median_ms, fill
= Benchmark)) + geom_col(show.legend = FALSE) +
  coord_flip() + facet_wrap(~Benchmark, scales = "free", ncol = 1) +
scale_y_log10() +
  labs(title = "Median execution time per approach", x = NULL, y = "Median time
(ms, log scale)")
```






```
ok <- tryCatch({
  for (p in c("sf", "leaflet")) if (!requireNamespace(p, quietly = TRUE))
install.packages(p)
  library(sf); library(leaflet)
  download.file(file.path(base_url, "misc/taxi_zones.zip"), "data/taxi_zones.zip",
mode = "wb", quiet = TRUE)
  unzip("data/taxi_zones.zip", exdir = "data/taxi_zones")
  shp <- st_read(list.files("data/taxi_zones", pattern = "\\.shp$", full.names =
TRUE, recursive = TRUE), quiet = TRUE)
  shp <- st_transform(shp, 4326)

  zone_stats <- dt[, .(pickups = .N, revenue = sum(total_amount), avg_fare =
mean(fare_amount)), by = PULocationID]
  shp <- merge(shp, zone_stats, by.x = "LocationID", by.y = "PULocationID", all.x
= TRUE)
  pal <- colorNumeric("YlOrRd", domain = log10(shp$revenue + 1), na.color =
"#dddddd")

  leaflet(shp) %>% addProviderTiles("CartoDB.Positron") %>%
    addPolygons(fillColor = ~pal(log10(revenue + 1)), fillOpacity = 0.75, weight =
0.6, color = "white",
                label = ~sprintf("%s (%s): %s pickups, $%s revenue, avg fare
$%.2f", zone, borough,
                                 comma(pickups), comma(round(revenue)), avg_fare))
%>%
    addLegend(pal = pal, values = ~log10(revenue + 1), title = "log10(revenue,
USD)")
}, error = function(e) { message("Leaflet section skipped: ",
conditionMessage(e)); NULL })
ok
```

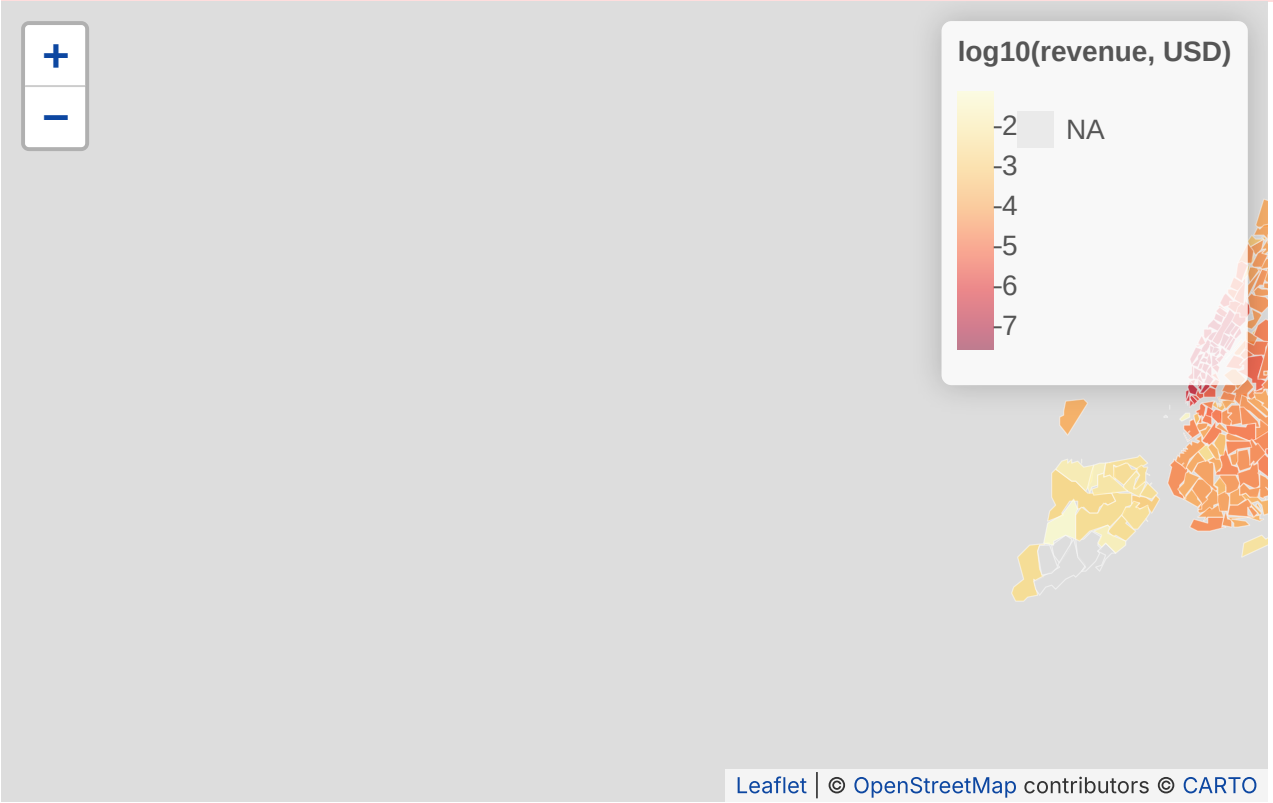
```
Installing package into ‘/usr/local/lib/R/site-library’
(as ‘lib’ is unspecified)

also installing the dependencies ‘proxy’, ‘e1071’, ‘wk’, ‘classInt’, ‘s2’, ‘units’


Installing package into ‘/usr/local/lib/R/site-library’
(as ‘lib’ is unspecified)

also installing the dependencies ‘lazyeval’, ‘sp’, ‘terra’, ‘crosstalk’, ‘leaflet.
providers’, ‘png’, ‘raster’


Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE
```




```r
fastest <- function(b) results[Benchmark == b][which.min(Median_ms)]
slowest <- function(b) results[Benchmark == b][which.max(Median_ms)]
for (b in unique(results$Benchmark)) {
  f <- fastest(b); s <- slowest(b)
  cat(sprintf("%-42s fastest: %-40s (%9.1f ms) | slowest: %-40s (%9.1f ms) | ratio
%.1fx\n",
              b, f$Approach, f$Median_ms, s$Approach, s$Median_ms, s$Median_ms /
f$Median_ms))
}
cat(sprintf("\nParallel speedup (heavy task, %d workers): %.2fx; light task:
%.2fx\n",
            n_cores, t_seq / t_par, t_seq_light / t_par_light))
```

```
A: mean fare by hour (full data)           fastest: data.table
(    347.4 ms) | slowest: base aggregate()                          (   3663.1 ms)
| ratio 10.5x
B: tip % per trip (100k rows)              fastest: vectorized
(      0.3 ms) | slowest: functional apply()                        (    519.2 ms)
| ratio 1790.2x
C: filter + group pipeline (full data)     fastest: data.table pipeline
(    396.4 ms) | slowest: base R pipeline                           (   2469.0 ms)
| ratio 6.2x
D: bootstrap per month (3 partitions)      fastest: sequential lapply
(  10419.0 ms) | slowest: parallel foreach/doParallel (2 workers)  (  11649.0 ms)
| ratio 1.1x
E: light task (24 partitions)              fastest: sequential lapply
(     27.0 ms) | slowest: parallel foreach/doParallel (2 workers)  (    823.0 ms)
| ratio 30.5x

Parallel speedup (heavy task, 2 workers): 0.89x; light task: 0.03x
```